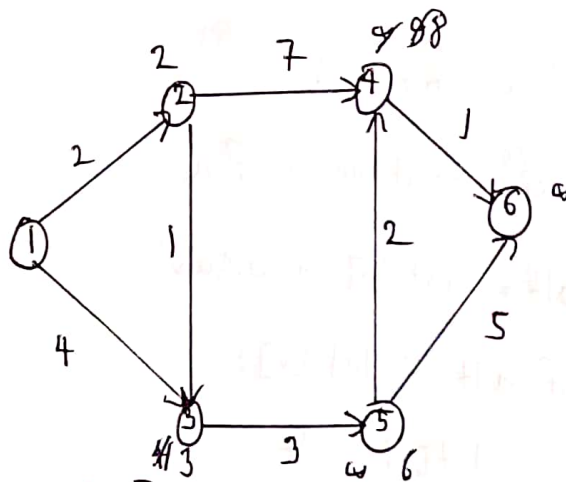


Qwz-4 AssignmentAssigned Topic: Dijkstra AlgorithmDescription:

Definition:

Dijkstra's Algorithm is a ^{single source} shortest path algorithm used in weighted graphs (with non-negative edge weights) to find the minimum distance from a source to all other nodes.

Diagram:

Relaxation:

$$\text{if } (d[u] + c(u,v) < d[v])$$

$$d[v] = d[u] + c(u,v)$$

select/ Vertex	1	2	3	4	5	6
1						
2	②	4	∞	∞	∞	
3	②	③	7	∞	∞	
5	②	③	9	⑥	∞	
4	②	③	⑧	⑥	11	
	②	③	⑧	⑥	9	

P.T.O

Pseudocode:

Dijkstra (G , source):

for each vertex v in G :

$dist[v] = INF$

$prev[v] = NULL$

$dist[source] = 0$

$\Theta =$ set of all vertices in G

while Θ is not empty:

$u =$ vertex in Θ with min $dist[u]$

remove u from Θ

for each neighbor v of u :

$alt = dist[u] + w(u, v)$

if $alt < dist[v]$:

$dist[v] = alt$

$prev[v] = u$

return $dist[]$, $prev[]$

Complexity Analysis:

Using Adjacency Matrix + array:

Time Complexity: $O(V^2)$

Space Complexity: $O(V)$

Using Adjacency list + min priority queue(heap):

Time complexity: $O((V+E) \log V)$

Space Complexity: $O(V+E)$

Proof of Correctness:

Goal of the Proof:

- Assume, S : set of nodes whose shortest distance are finalized. We need to show that when a node v is added to S :

$$d[v] = \delta(s, v)$$

- $d[v]$: Shortest distance to node v calculated by the algorithm,
- $\delta(s, v)$ is the ^{true} shortest path between from s to v .

Trivially we can say, the calculated distance can't be better than actual shortest distance: $d[v] \geq \delta(s, v)$ — eqn. 1

p.T.O

Proof outline (by Induction):

Base case: $d[s] = \delta(s, s) = 0$ (trivial)

Inductive step:

- Assume $d[u] = \delta(s, u)$ for all u in S
- Show that for the next node v selected: $d[v] = \delta(s, v)$

Let p be a shortest path from s to v , and let u be the last node on p such that $u \in S$ and the next node $w \notin S$

$$s \rightsquigarrow u \rightarrow w \rightarrow \dots \rightarrow v$$

Since $u \in S$, by inductive hypothesis:

$$d[u] = \delta(s, u)$$

- After relaxing (u, w) where $u \in S$ and (u, w) is on the shortest path.

$$d[w] \leq d[u] + w(u, w)$$

- since $d[u] = \delta(s, u)$, we get:

$$d[w] \leq \delta(s, u) + w(u, w)$$

- Since edge (u, w) is in the shortest path;

$$\delta(s, w) = \delta(s, u) + w(u, w)$$

$$\text{So, } d[w] \leq \delta(s, w) \text{ ————— eqn. 2}$$

- As the algorithm picks v , before w , so:

$$d[v] \leq d[w] \dots \dots \text{eqn. 3}$$

- Combining eqn. 2 and 3,

$$d[v] \leq \delta(s, w) \dots \dots \text{eqn. 4}$$

- Now, $\delta(s, v) = \delta(s, w) + \text{cost}(w, v)$ and every edge is non-negative.

so the cost will be greater or equal to 0. So:

$$\delta(s, v) \geq \delta(s, w)$$

- From eqn. 4:

$$d[v] \leq \delta(s, v) \dots \dots \text{eqn. 5}$$

We have proved that

$$d[v] \geq \delta(s, v) \dots \dots \text{eqn. 1}$$

$$d[v] \leq \delta(s, v) \dots \dots \text{eqn. 5}$$

Both equations can only be true if:

$$d[v] = \delta(s, v)$$

(Proved)

Q1 Application of the Algorithm;

Example:

Navigation system (like Google maps): uses dijkstra's algorithm to compute shortest for driving or walking distance.

Problem: Statement:

You are given an undirected graph with n vertices and m edges,

Let source, $s=1$ and $t=n$,

You need to find all the edges such that if you remove one that edge, the shortest path of distance from s to t becomes strictly larger (or there's no path)

Constraints:

- $n \leq 3 \times 10^5$
- $m \leq \min(3 \times 10^5, n(n-1)/2)$
- Edge weights are positive.

You need to output the list of such edges in increasing order